

Database Systems

Group 3

Nayla Pramesti Adhina
Yohana Indah N. Br. S
Rafael Raditya Setyono
Ryan Gazendra Irawan

Our Team

Tokoh papan atas, atas segalanya #hazek



Naypram cantik

CEO



Yoyo memukau

CEO



Fael pilates

CEO



Kocheng kicau mania

CEO

Project Overview

"Built to make gym administration simpler, faster, and smarter."

What is Gym-Ease?

GymEase is a full-stack web application designed to help gym administrators manage their daily operations efficiently from member registration and trainer scheduling to class bookings and attendance tracking.

Member Management

Register and Track gym members

Member Management

Register and Track gym members

Member Management

Register and Track gym members

Member Management

Register and Track gym members



Tech Stack

Frontend

REACT + VITE

SPA with JSX pages & component-based UI

Backend

Node.js + Express

REST API with JWT authentication & middleware

Database

PostgreSQL (Neon Cloud)

Serverless cloud DB via @neondatabase/serverless

Graph DB

Neo4j

Graph relationships between entities via neo4j-driver

Deployment

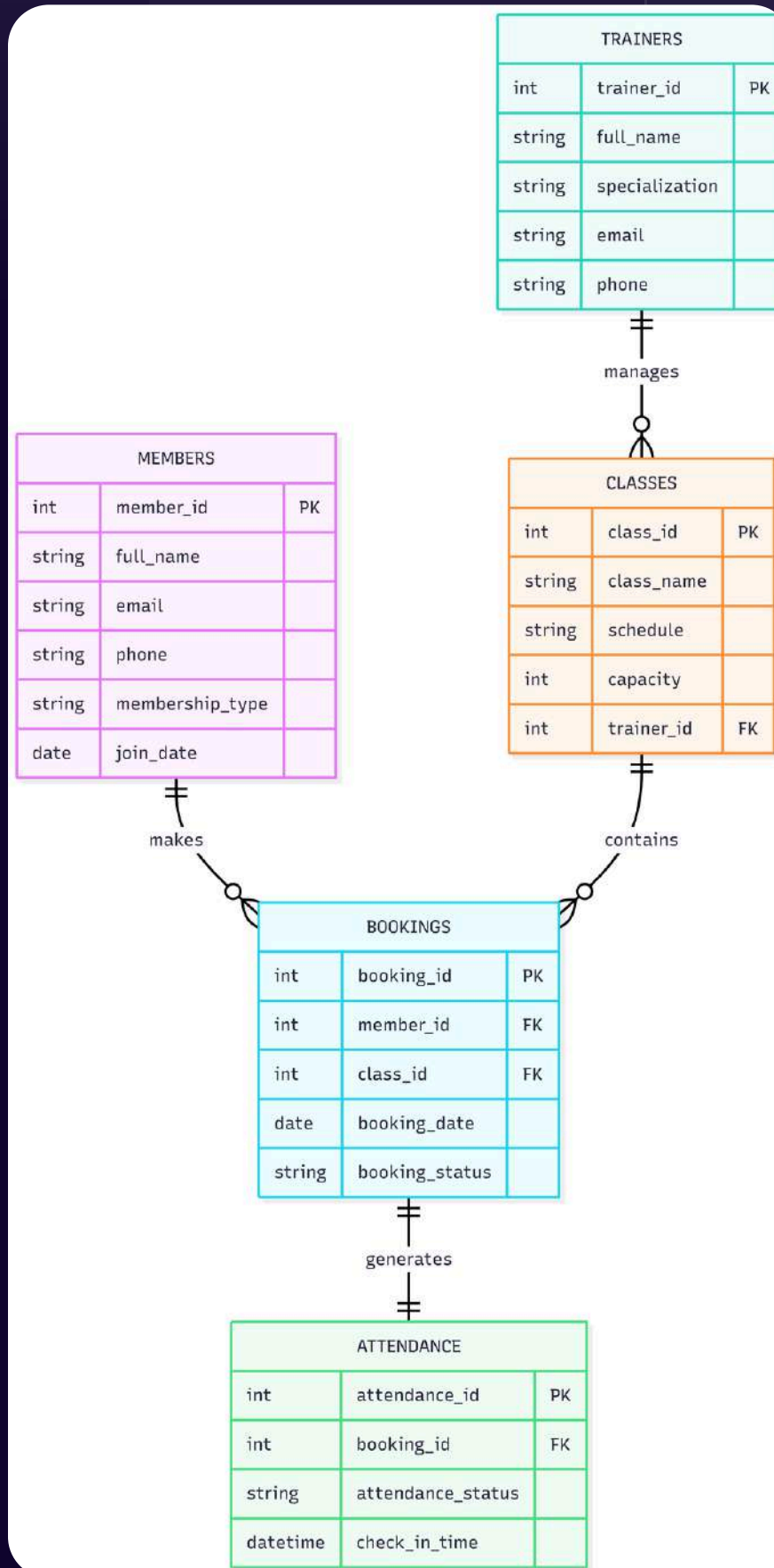
Railway

Cloud deployment with auto-deploy from GitHub



ER Diagram

The ERD describes the structure of the GymEase database system and the relationships between entities.



Members

The MEMBERS table stores all gym member information such as name, email, phone number, membership type, and join date.

Trainers

The TRAINERS table contains trainer information including specialization and contact details.

Classes

The CLASSES table stores fitness class data such as class name, schedule, and maximum capacity. Each class is managed by one trainer through the trainer_id foreign key.

Bookings

The BOOKINGS table acts as a bridge between members and classes. It records which member booked which class and stores booking status information.

Attendance

The ATTENDANCE table records whether a booked member attended the class and stores the check-in time.

Relationships

- One member can make many bookings.
- One class can have many bookings.
- One trainer can manage many classes.
- One booking generates one attendance record.

UML Class Diagram

The UML Class Diagram illustrates the object-oriented structure of the GymEase application.

Member Class

Represents gym members who can register, book classes, and cancel bookings.

GymClass Class

Represents fitness classes with schedules and capacity management functions.

Attendance Class

Stores attendance records and manages member check-in status.

Trainer Class

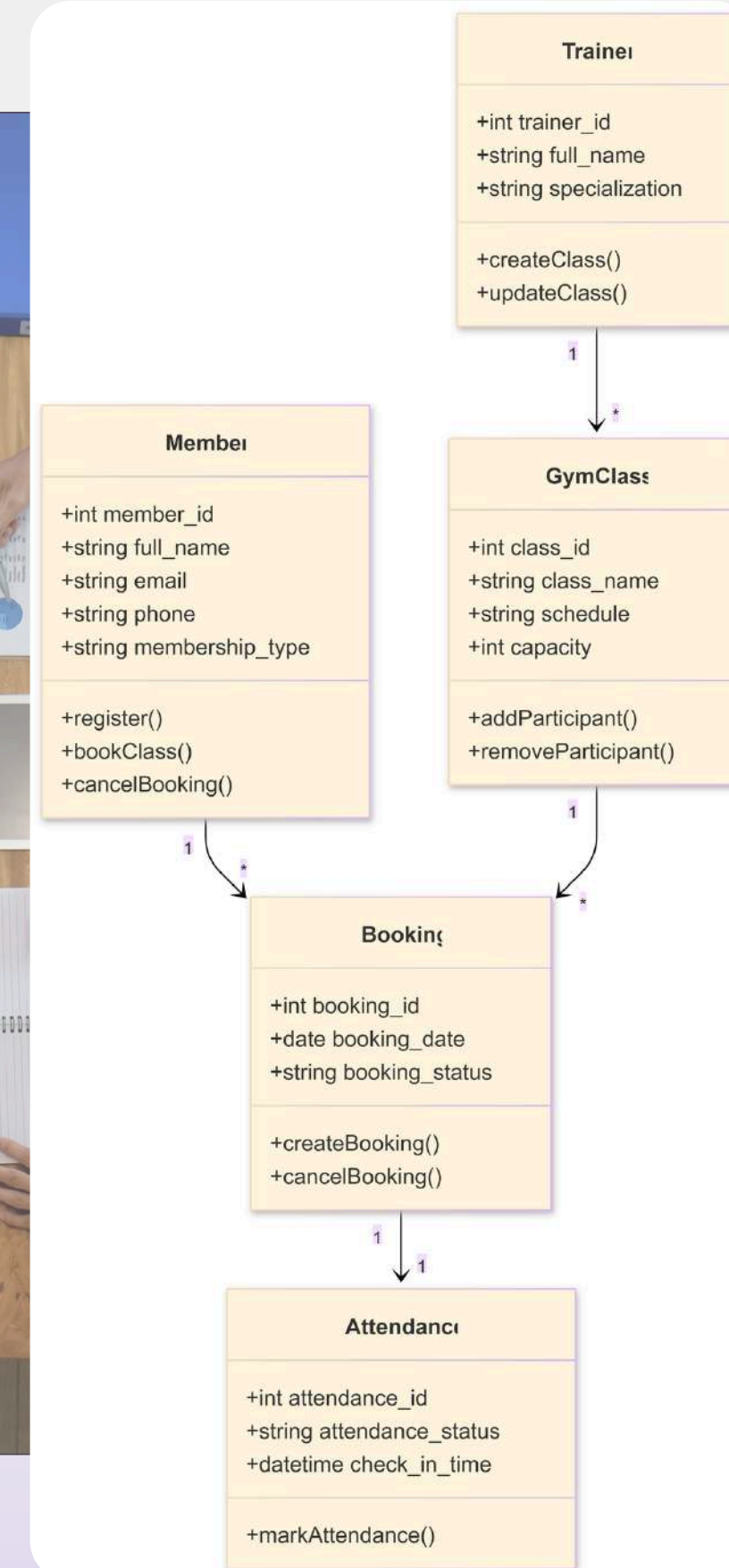
Represents trainers responsible for creating and managing gym classes.

Booking Class

Handles the booking process between members and gym classes.

Relationships

- A member can have multiple bookings.
- A gym class can contain multiple bookings.
- A trainer can manage multiple gym classes.
- Each booking has exactly one attendance record.



Flowchart

The flowchart explains the operational workflow of the GymEase application.

User Authentication

The process begins when the user opens the application and logs in. If login fails, the system displays an error message and returns to the login page.

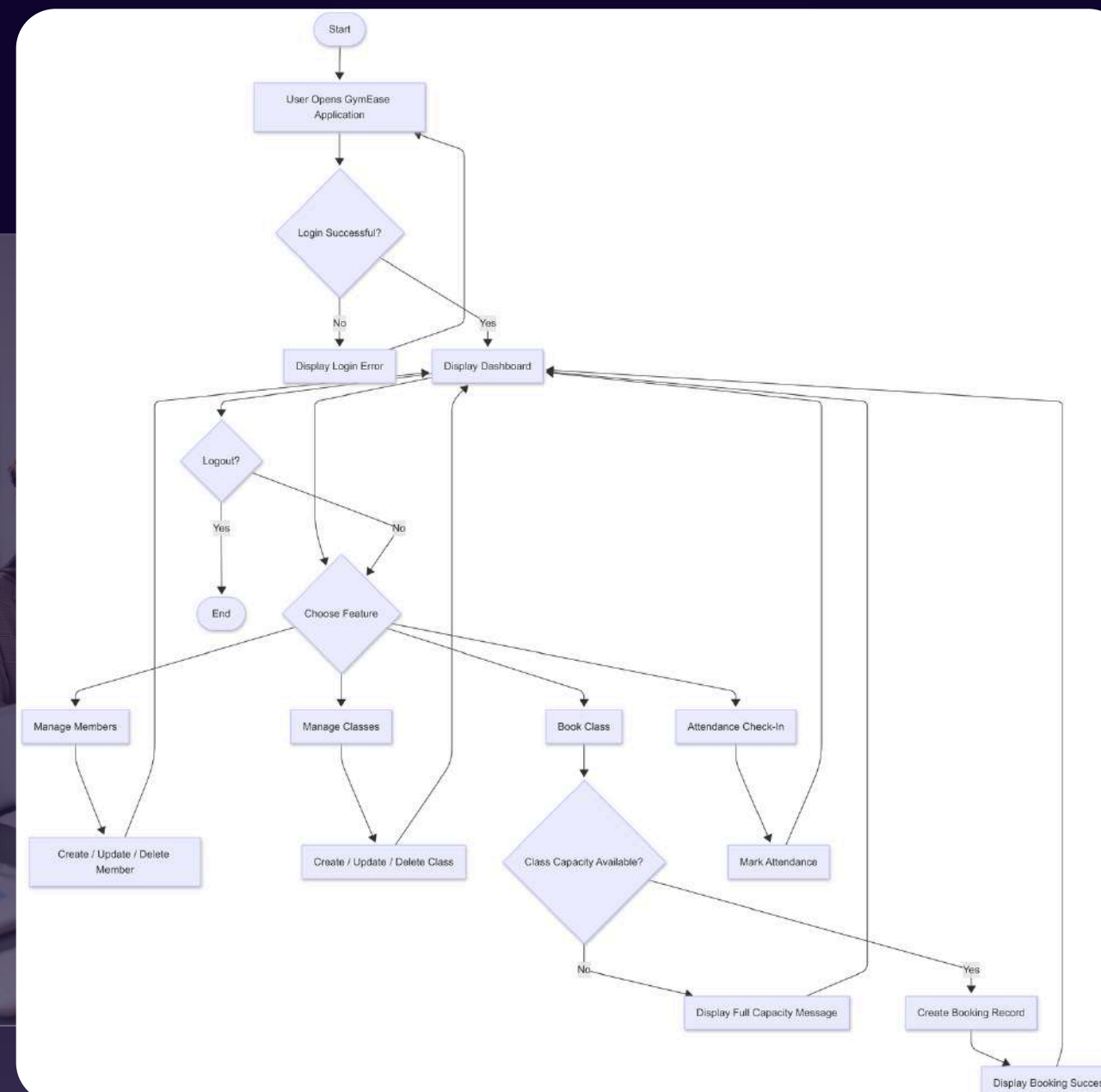
Feature Selection

The user can:

- Manage members
- Manage classes
- Book classes
- Perform attendance check-in

Attendance Process

Users or staff can mark attendance for booked members.



Dashboard Access

After successful login, the user accesses the main dashboard.

Booking Process

When booking a class:

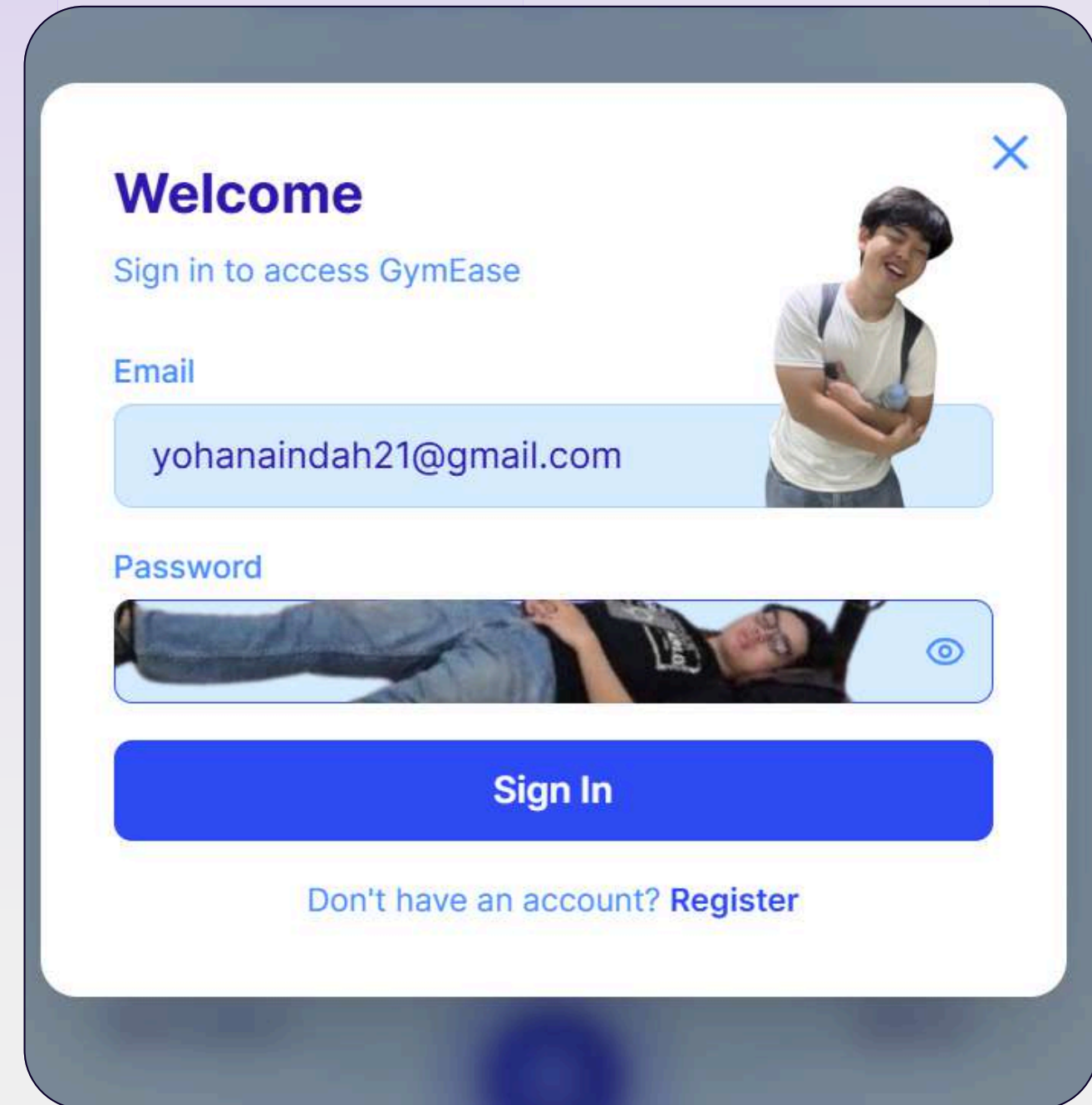
- The system checks class capacity.
- If the class is full, an error message is displayed.
- If capacity is available, the booking is created successfully.

Logout

The user may continue using features or logout to end the session.

Display &
Log In

Features

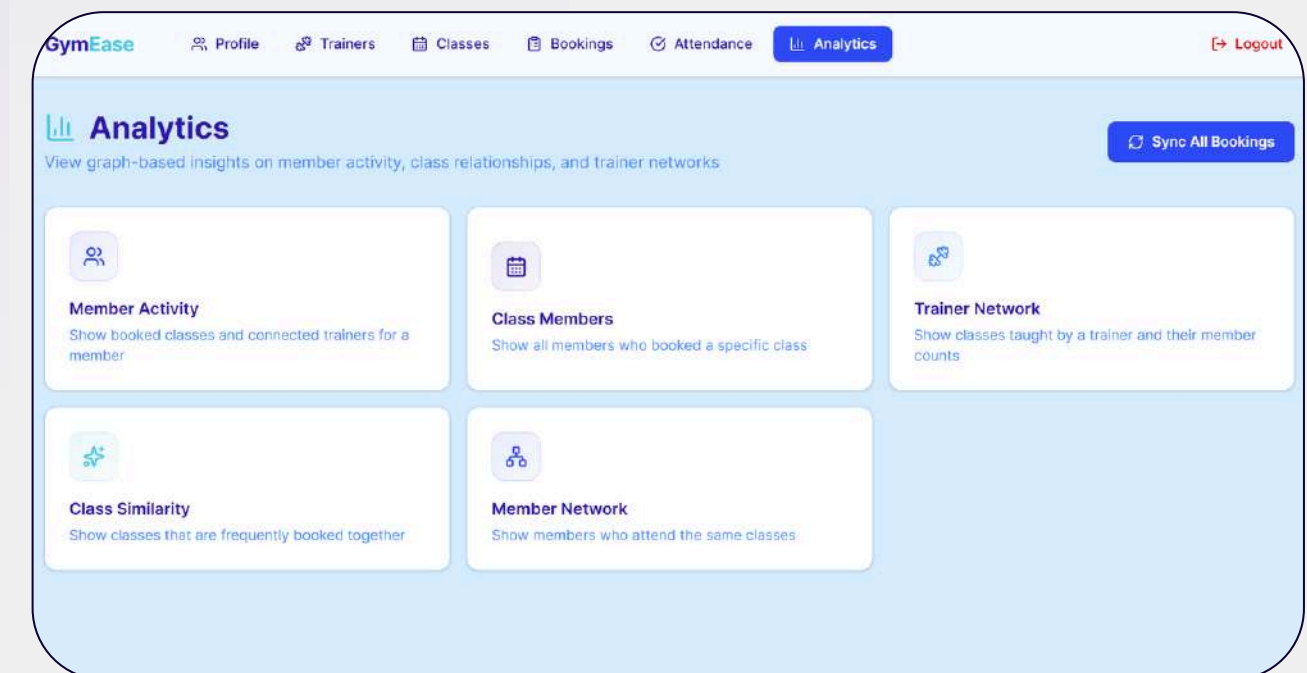
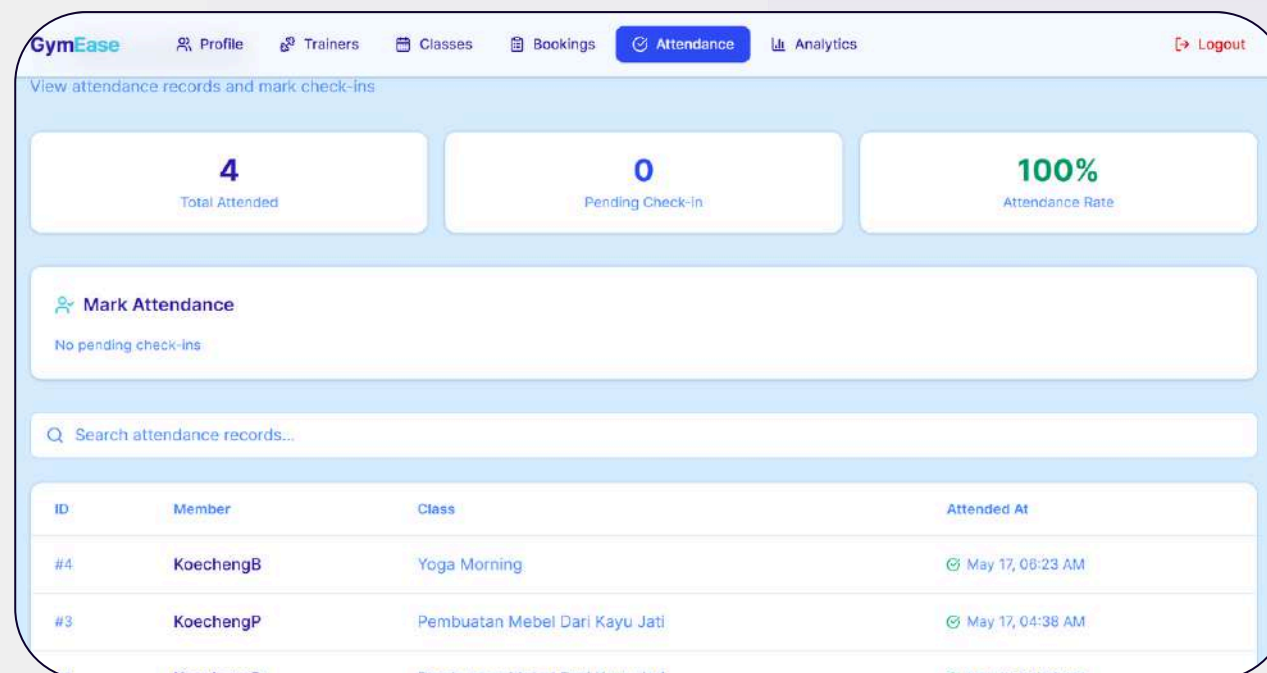
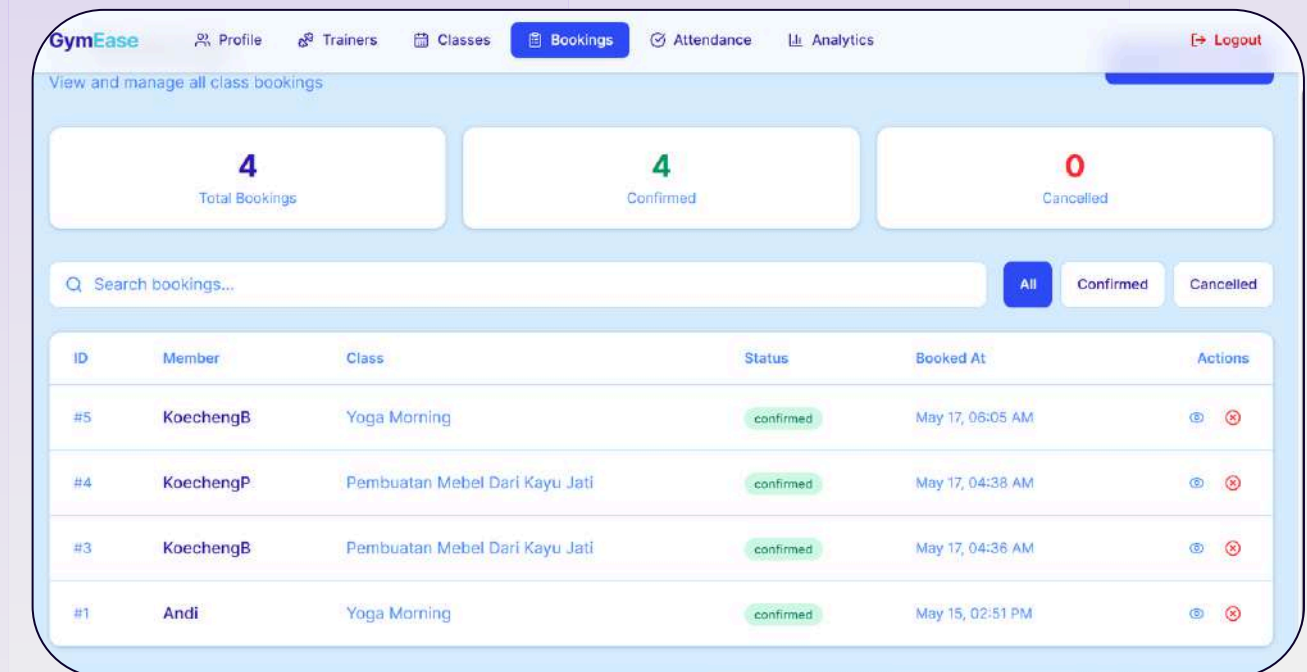
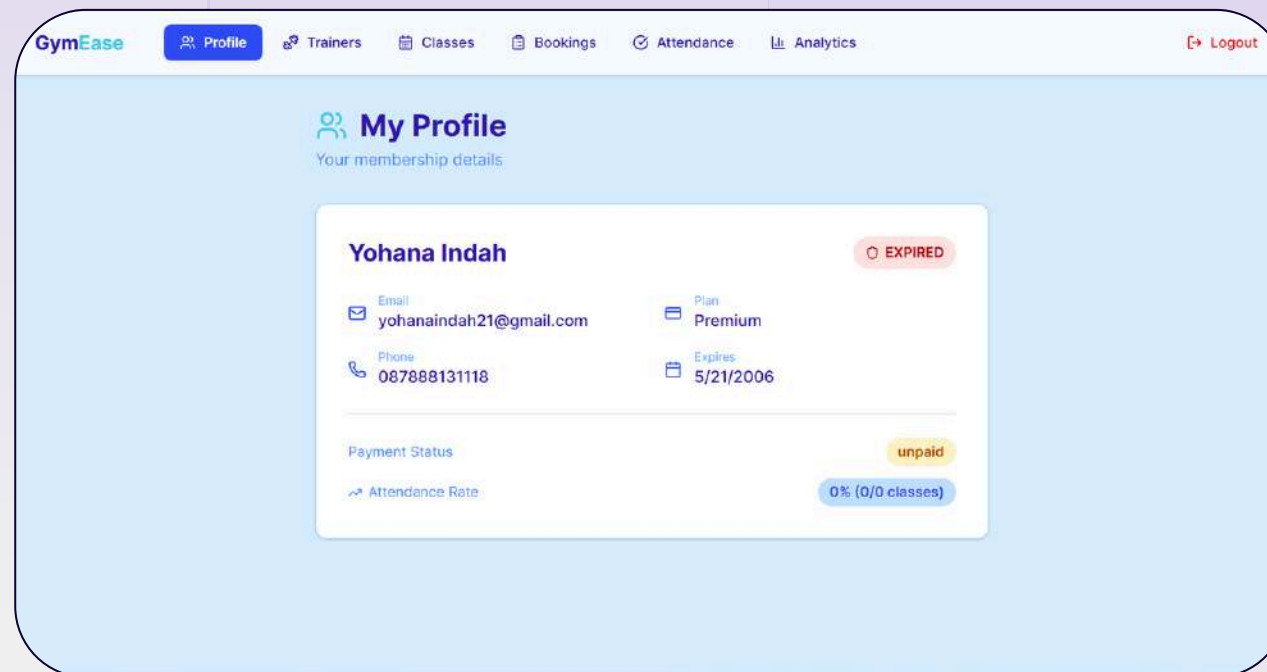


The login screen displays a "Welcome" message and a prompt to "Sign in to access GymEase". It includes input fields for "Email" (containing "yohanaindah21@gmail.com") and "Password" (with a toggle icon). A blue "Sign In" button is positioned below the fields. At the bottom, a link reads "Don't have an account? Register". A close button (X icon) is located in the top right corner. A background image of a smiling man is visible on the right side of the form.

Secure JWT-based authentication. Register, login, and protected API routes via middleware.

Member

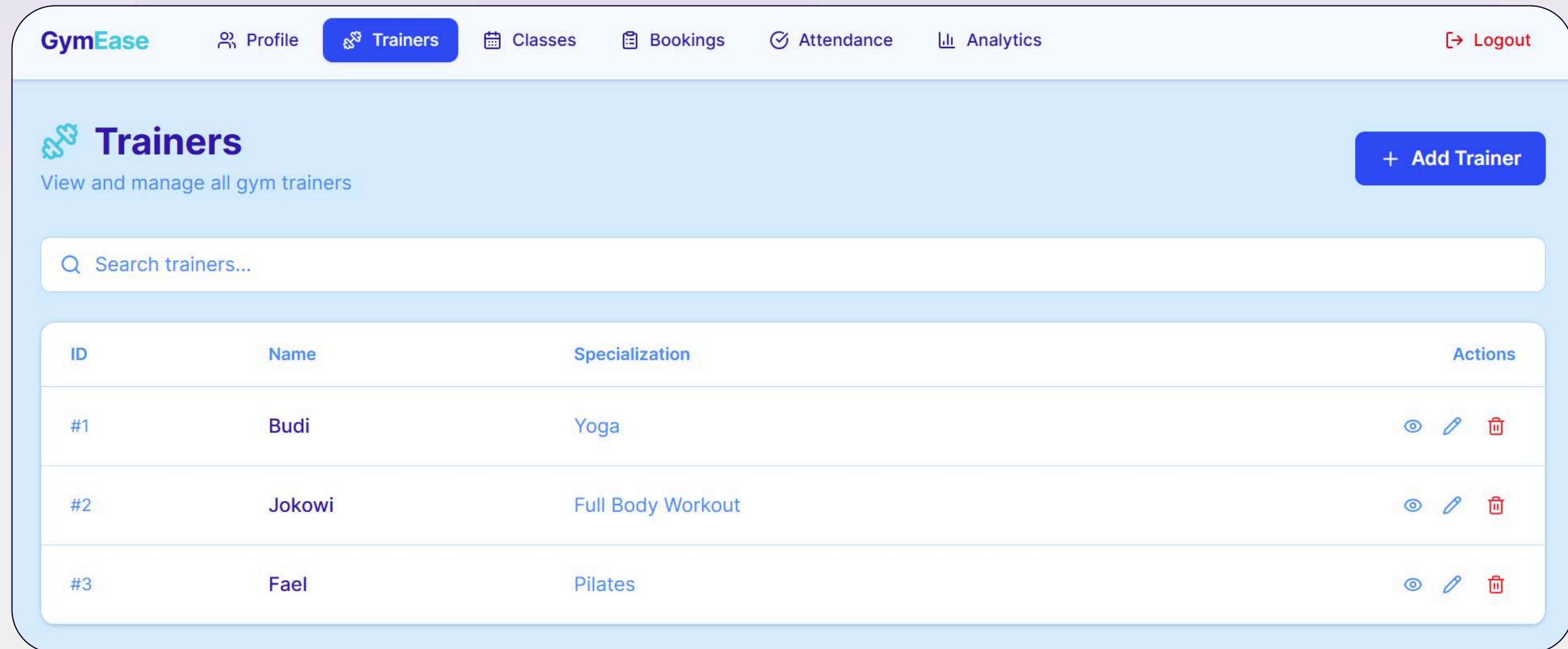
Features



Add, view, and manage gym members. Each member can be tracked across bookings and attendance.

Trainer

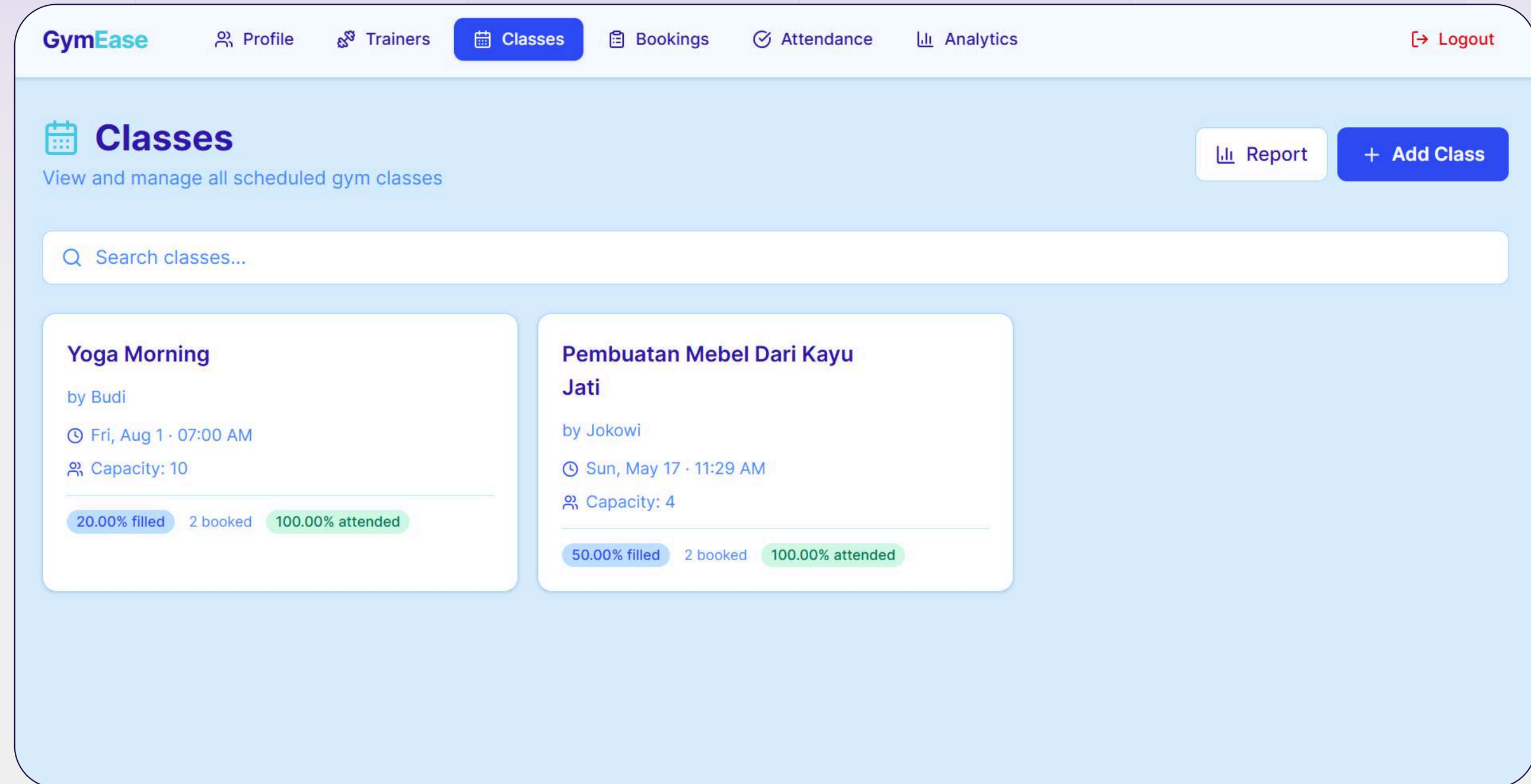
Features



Manage trainer profiles including specializations and contact info.

Classes

Features

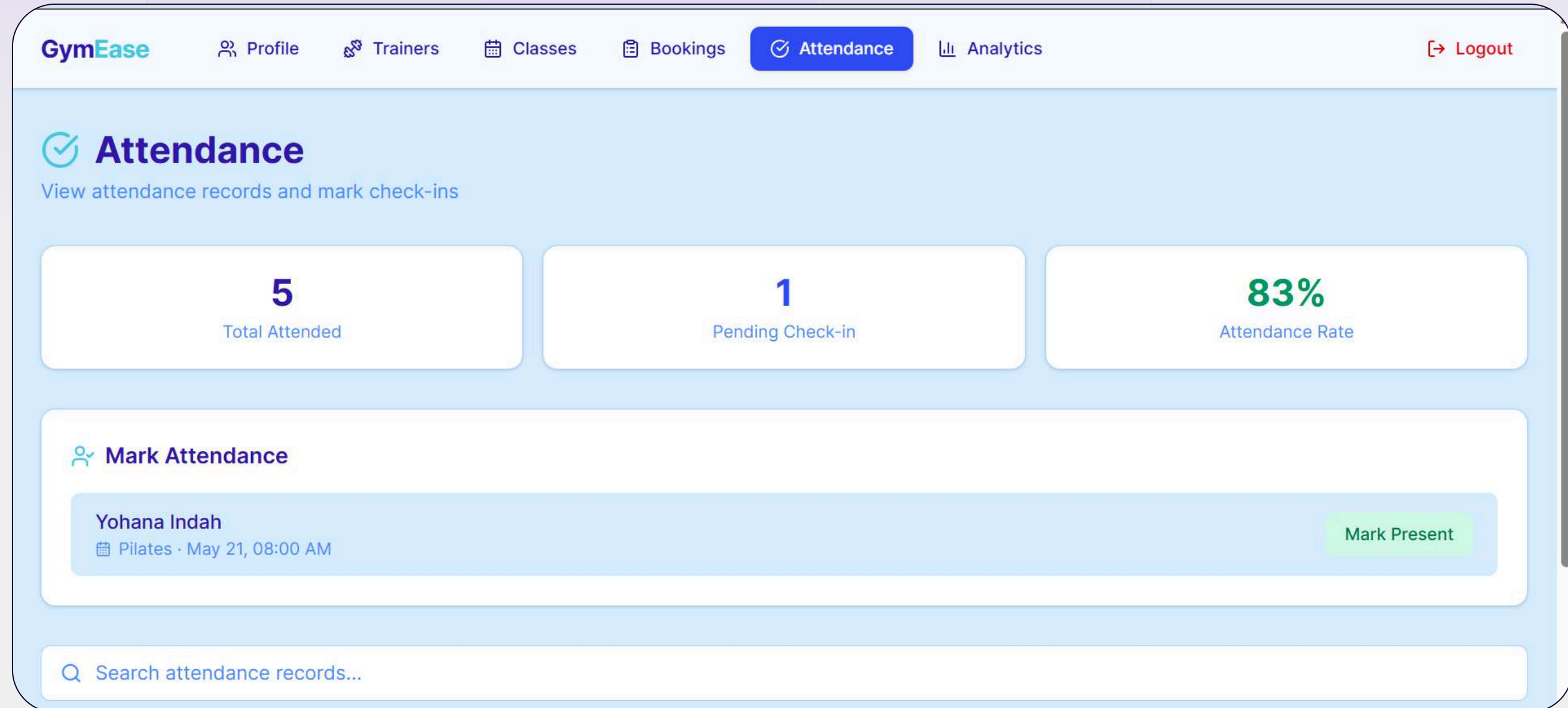


Create and manage fitness classes with capacity limits, schedules, and trainer assignments.



Attendance

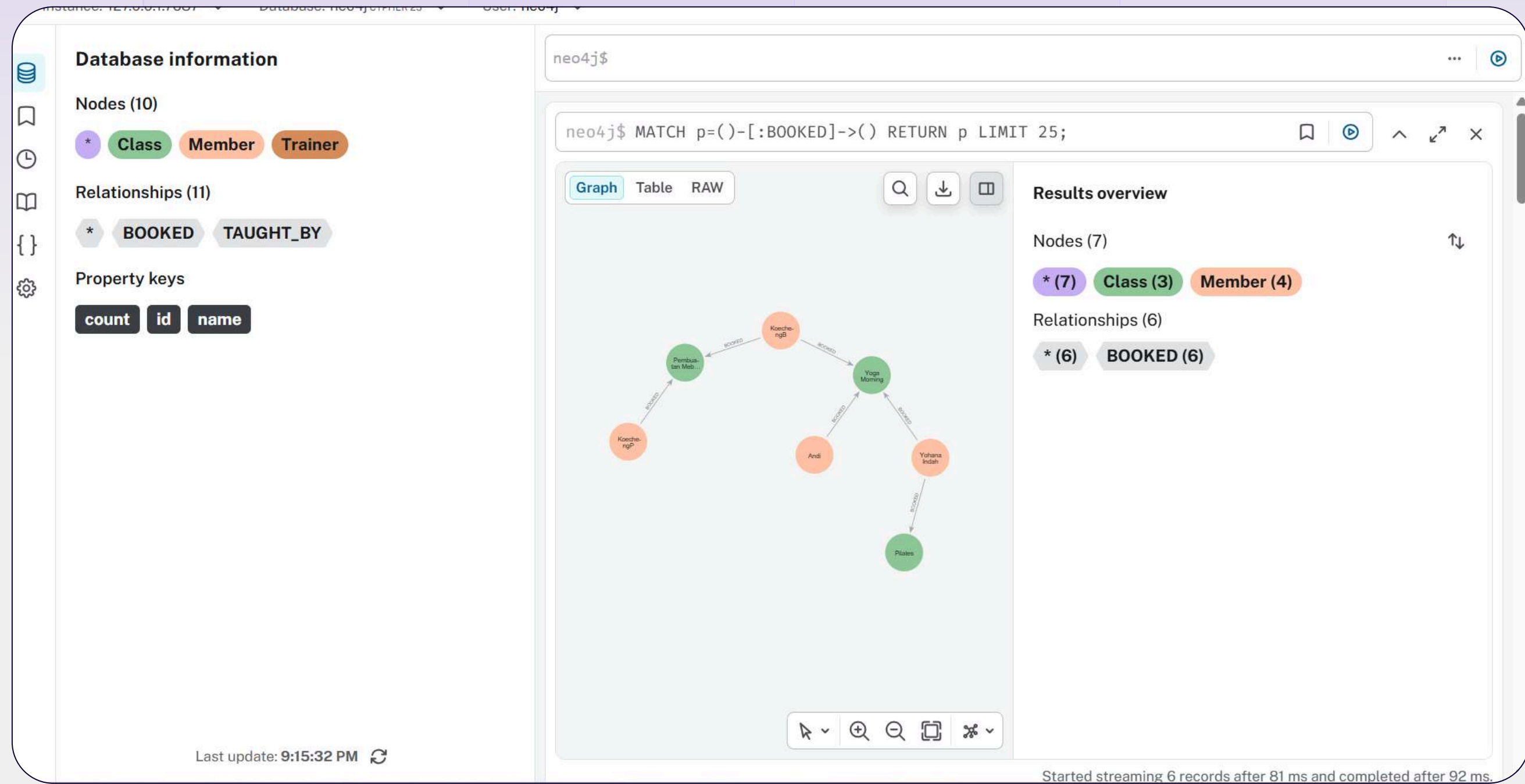
Features



Record and view member attendance per class session.

Graph

Features



Neo4j models connections between members, classes, and trainers for advanced querying.

API Endpoints

/api/auth

POST /register, POST /login
User authentication & JWT token issuance

/api/trainers

GET, POST, PUT, DELETE
CRUD operations for trainer profiles

/api/classes

GET, POST, PUT, DELETE
Manage fitness class schedules & capacity

/api/bookings

GET, POST, PUT, DELETE
Member class booking management

/api/auth

POST /register, POST /login
User authentication & JWT token issuance

/api/trainers

GET, POST, PUT, DELETE
CRUD operations for trainer profiles

Conclusion & What We Learned

Integrating two databases (PostgreSQL + Neo4j) in one backend

Deploying a Node.js backend to Railway with env variables

Implementing JWT authentication across REST API routes

Collaborating with Git — dividing files & coordinating pushes

Structuring a full-stack project with clean separation of concerns

Thank you!

